

Ph. D. Juan David Martínez Vargas
Ph. D. Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

Lecture 06

Deep Learning

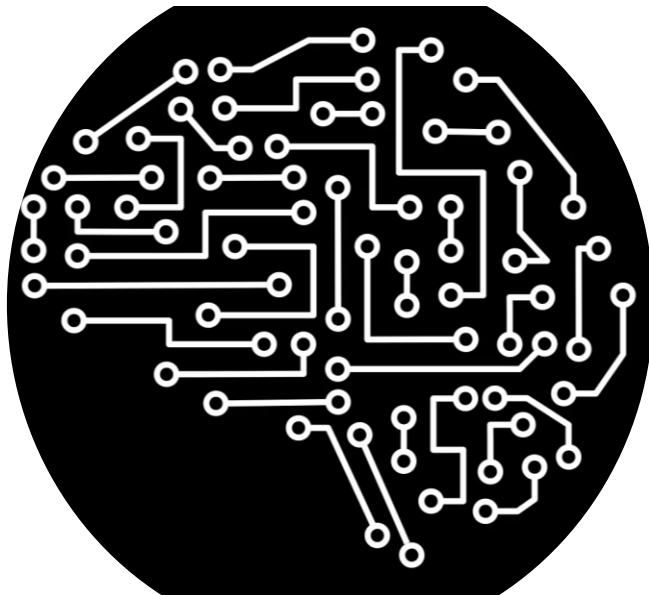
Agenda

- El problema de optimizar
- Learning Rate Decay
- Momentum
- Adagrad – RMSprop
- Adam

Optimizadores

Luego de calcular la pérdida e implementar backpropagation para calcular los gradientes la pregunta natural es ¿qué hace un modelo con esta información?”

El optimizador: un algoritmo que ajusta los pesos y los sesgos de una red neuronal durante el entrenamiento. Su función principal es cambiar estos valores para reducir al mínimo el error medido por la función de pérdida, logrando así que el modelo haga predicciones más exactas.



- SGD (Descenso de Gradiente Estocástico)
- SGD con Momento
- AdaGrad
- RMSprop
- Adam (Adaptive Moment Estimation)

Optimizadores

Backpropagation calcula los gradientes; el **optimizador** usa esos gradientes para actualizar los pesos y sesgos; el learning rate determina qué tan grande es esa actualización.

Hiperparámetros que controlan al optimizador: learning rate η , momentum, weight decay y parámetros específicos según el algoritmo. Por ejemplo, Adam tiene además β_1 , β_2 y ϵ .

$$\beta^{(k+1)} \leftarrow \beta^{(k)} - \eta \frac{\partial J}{\partial \beta}$$

Descenso de Gradiente Estocástico

```
Result:  $\mathbf{w}, b$ 
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$ 
for  $t = 1, \dots, T$  do
  for  $j = 1, \dots, n/k$  do
     $\Delta \mathbf{w} := 0, \Delta b := 0;$ 
    for  $\{(\mathbf{x}^{[i]}, y^{[i]}), \dots, (\mathbf{x}^{[i+k]}, y^{[i+k]})\} \subset D$  do
      cálculo de la salida;
      cálculo del error ;
      actualización de  $\Delta \mathbf{w}, \Delta b;$ 
    end
    actualización  $\mathbf{w}, b;$ 
     $\mathbf{w} := \mathbf{w} + \Delta \mathbf{w};$ 
     $b := b + \Delta w;$ 
  end
end
```

Descenso de Gradiente Estocástico

Minibatch learning y ruido en el gradiente es una forma de SGD, donde en cada paso usamos una muestra (lote) de los datos, no todo el conjunto, esto genera la **presenta cierta variabilidad (ruido)**.

Efecto potencialmente favorable: esta variabilidad puede favorecer la exploración de la función de pérdida y ayudar a salir de algunas regiones problemáticas, como mínimos locales o puntos de silla.

Efecto desfavorable: una variabilidad excesiva puede producir oscilaciones y cambios frecuentes en la dirección de actualización, haciendo la convergencia menos estable o más lenta.

El gradiente usando **todo el dataset**

$$g(\theta) = \frac{1}{N} \sum_{i=1} \nabla_{\theta} L_i(\theta)$$

El gradiente usando **un lote de dataset**

$$g_{mb}(\theta) = \frac{1}{N_{mb}} \sum_{i \in MB} \nabla_{\theta} L_i(\theta)$$

¿Podemos reducir las oscilaciones sin perder las ventajas del entrenamiento con mini-batches?

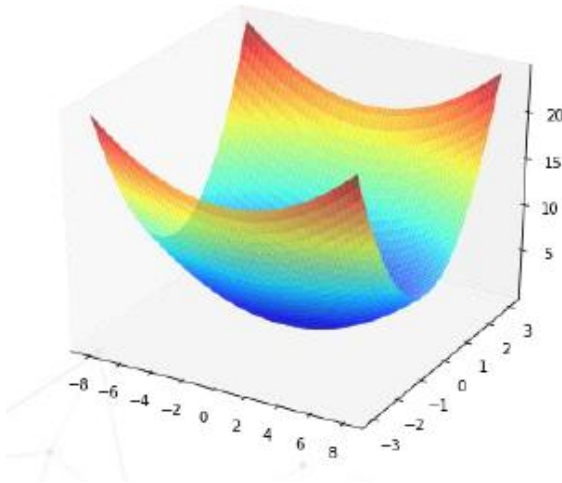
Descenso de Gradiente Estocástico

¿Podemos reducir las oscilaciones sin perder las ventajas del entrenamiento con mini-batches?

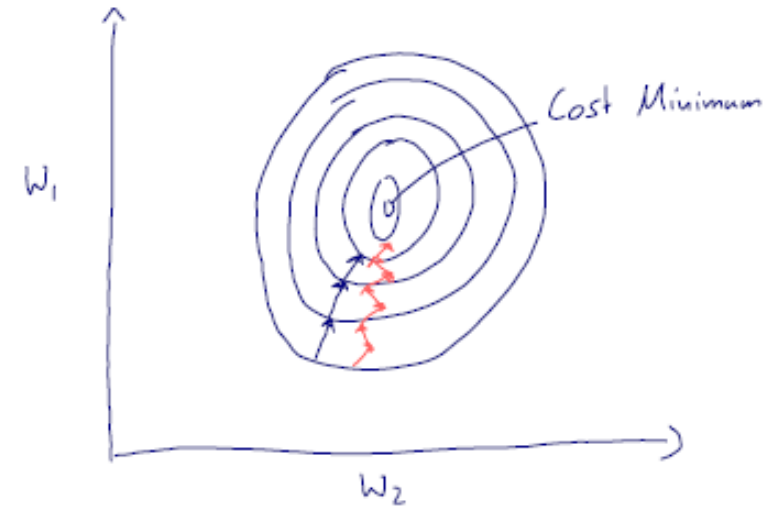
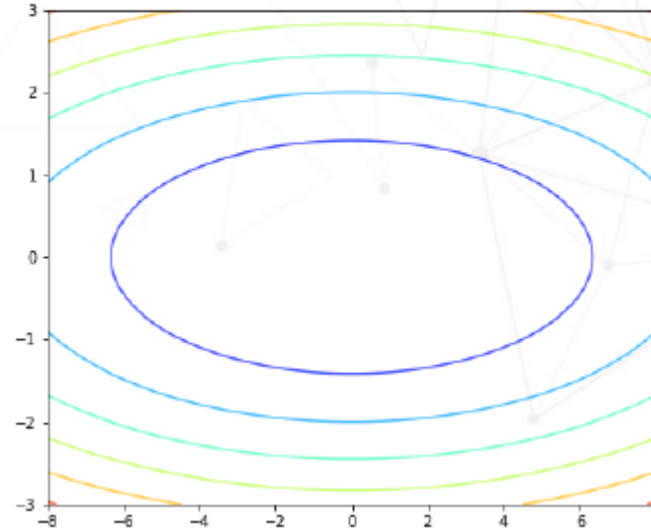
$$g(\theta) = \frac{1}{N} \sum_{i=1} \nabla_{\theta} L_i(\theta)$$

$$g_{\text{mb}}(\theta) = \frac{1}{N_{\text{mb}}} \sum_{i \in \text{MB}} \nabla_{\theta} L_i(\theta)$$

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$$



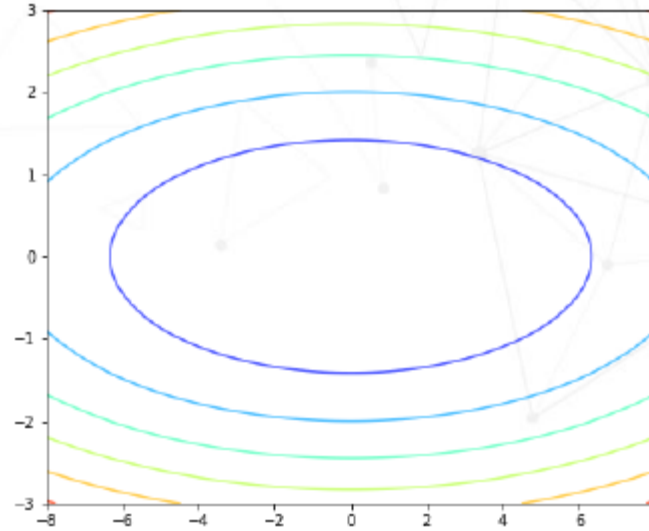
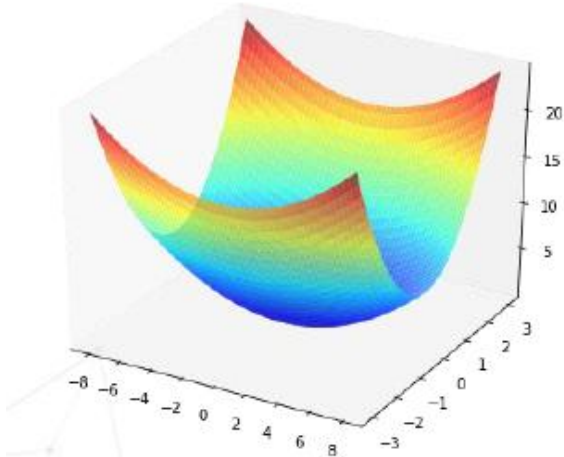
$$\frac{\partial f(x_1, x_2)}{\partial x_1} \ll \frac{\partial f(x_1, x_2)}{\partial x_2}$$



Función cuadrática anisotrópica – “compleja”

Descenso de Gradiente Estocástico

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$$

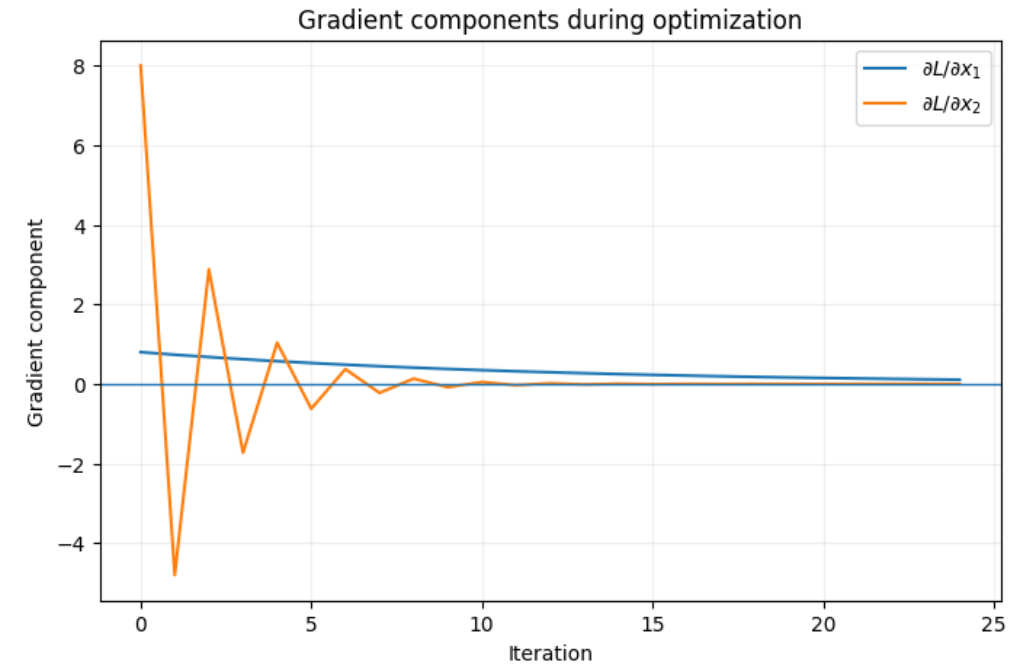


Función cuadrática anisotrópica – “compleja”

$$\nabla \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial x_1} \\ \frac{\partial \mathcal{L}}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 0.2x_1 \\ 4x_2 \end{bmatrix}$$

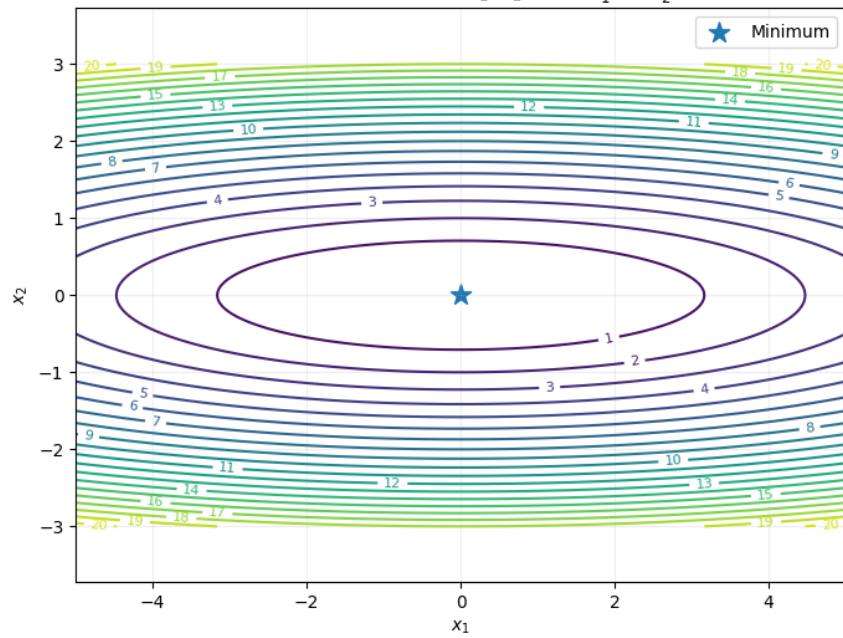
$$(x_1, x_2) = (1, 2) \rightarrow \begin{matrix} 0.2(1) = 0.2 \\ 4(1) = 4 \end{matrix}$$

$$\frac{\partial f(x_1, x_2)}{\partial x_1} \ll \frac{\partial f(x_1, x_2)}{\partial x_2}$$

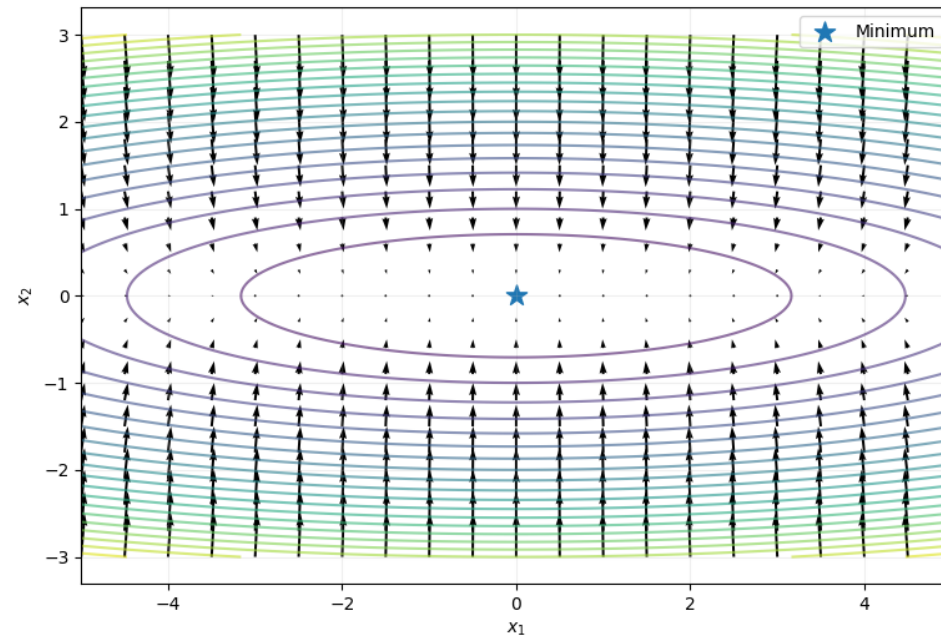


- Why not simply use a smaller learning rate?
- Can we reduce oscillations without making learning unnecessarily slow?

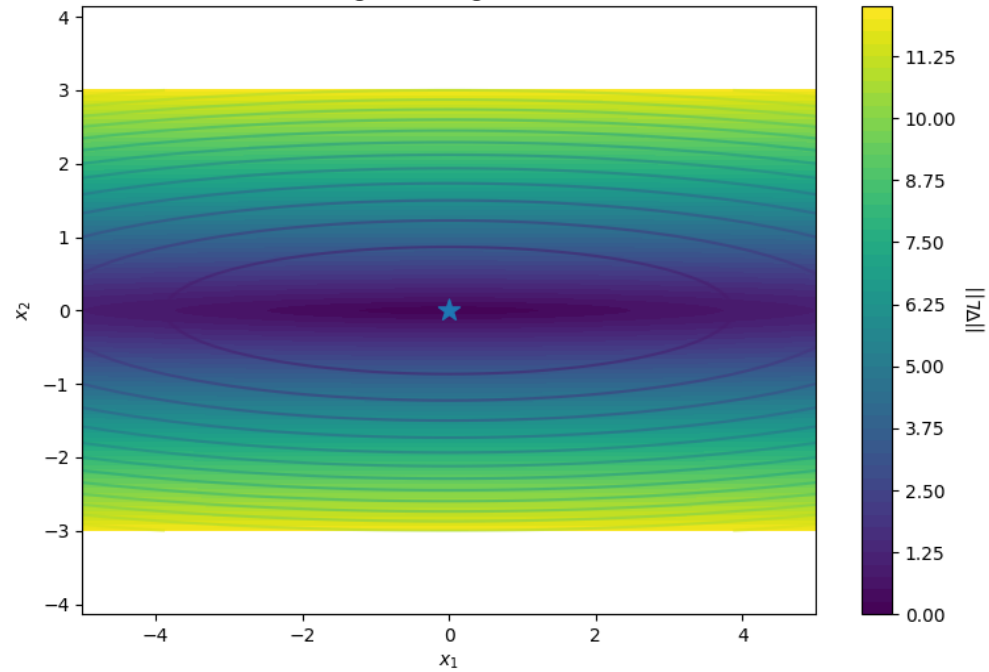
Curvas de nivel de $L(x_1, x_2) = 0.1x_1^2 + 2x_2^2$



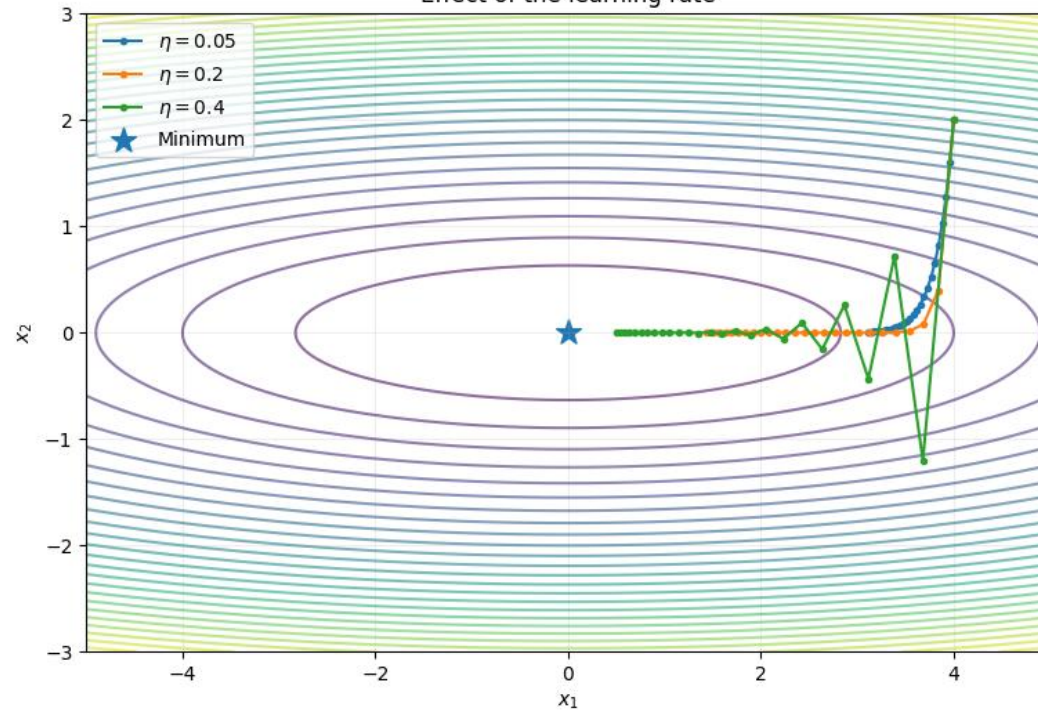
Dirección del descenso: $-\nabla L$



Magnitud del gradiente



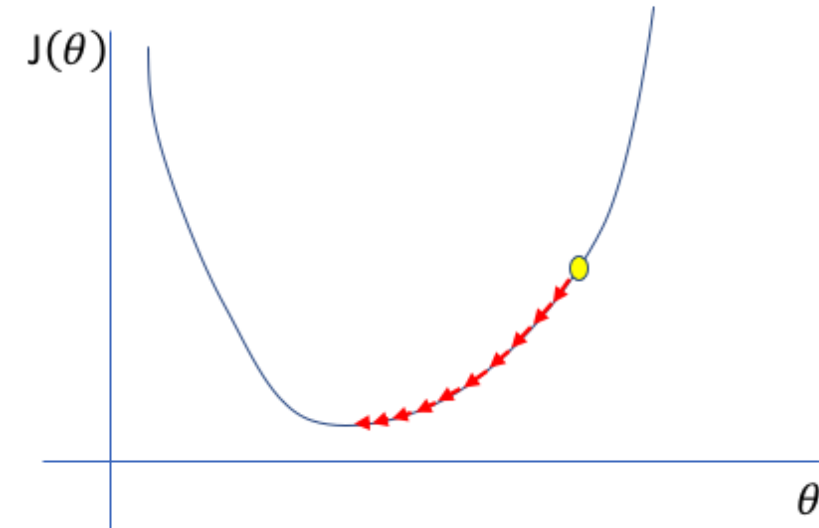
Effect of the learning rate



Optimizadores

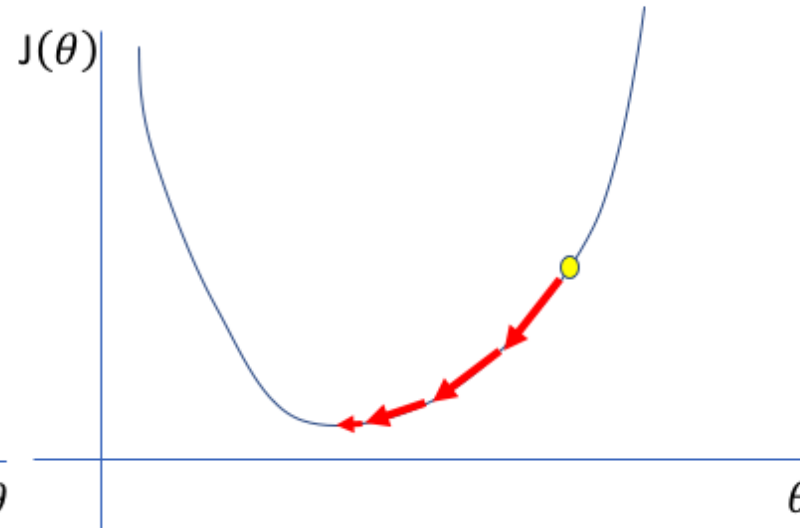
¿Quién controla la tasa de aprendizaje?

Too low



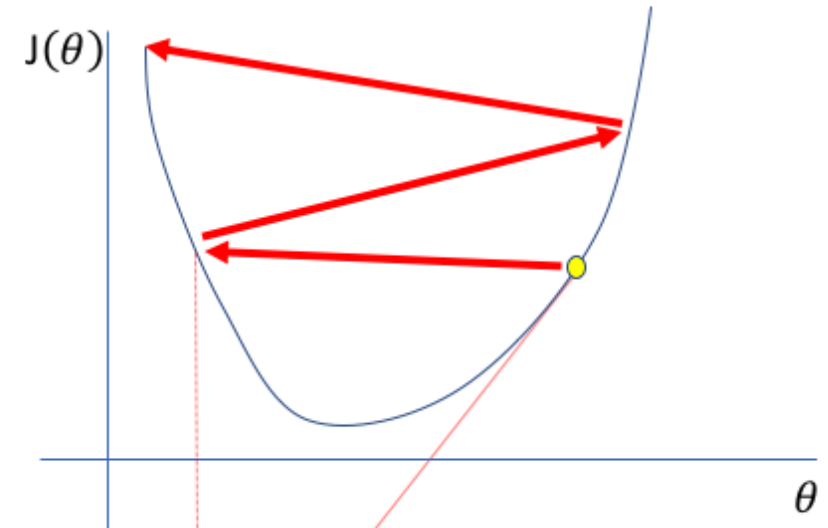
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

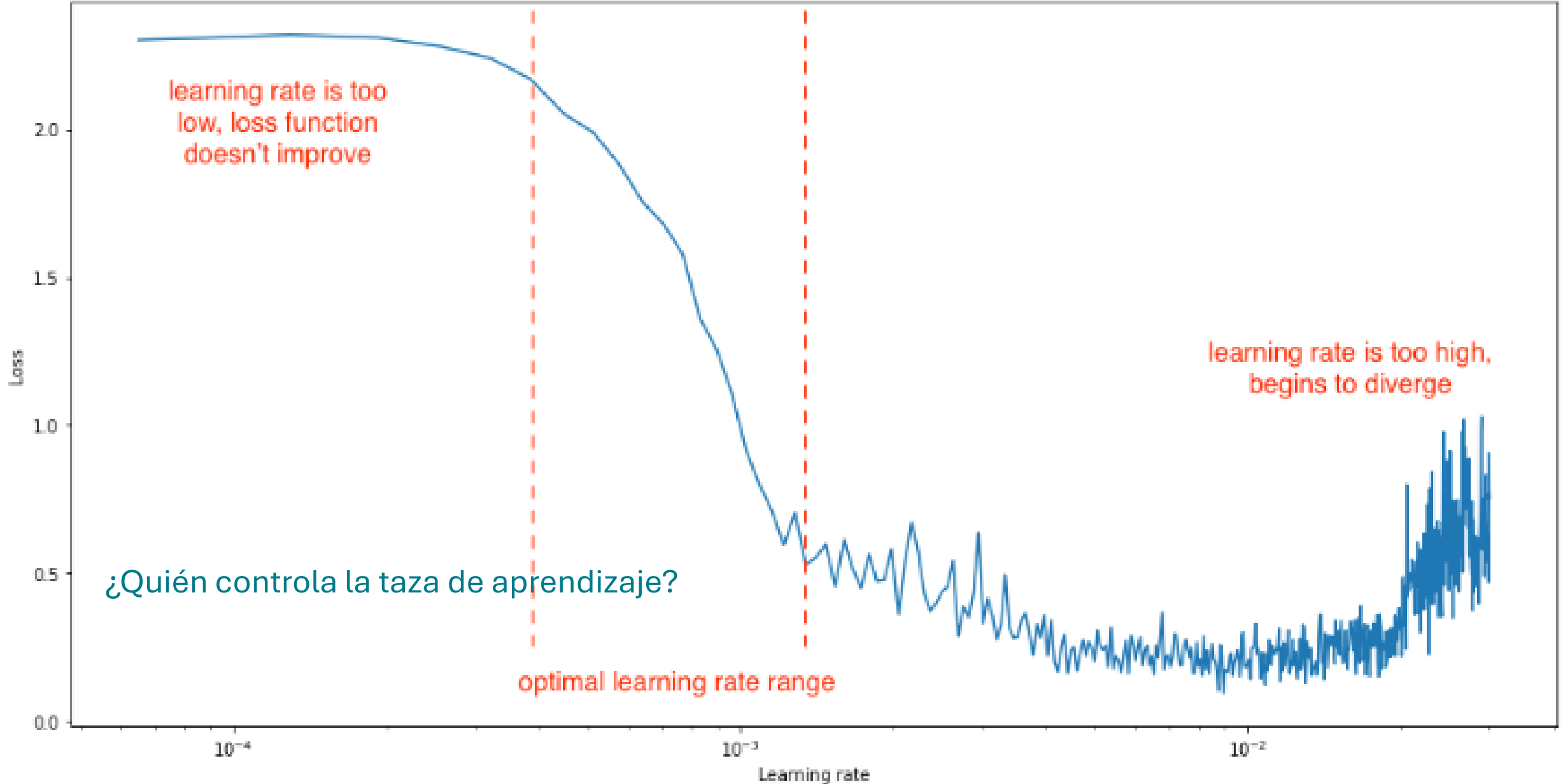
Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

[Link](#)

Optimizadores



Momentum

En lugar de usar solo el gradiente actual, ¿qué pasaría si también consideramos la dirección de las actualizaciones anteriores?

$$\theta = \{w_1, w_2, w_3 \dots w_n\}$$

Gradiente calculado
mediante backpropagation

Gradiente actual

$$g_t = \nabla_{\theta} L(\theta_t)$$

Controla cuánto contribuye
el gradiente actual

Learning rate

$$\eta g_t$$

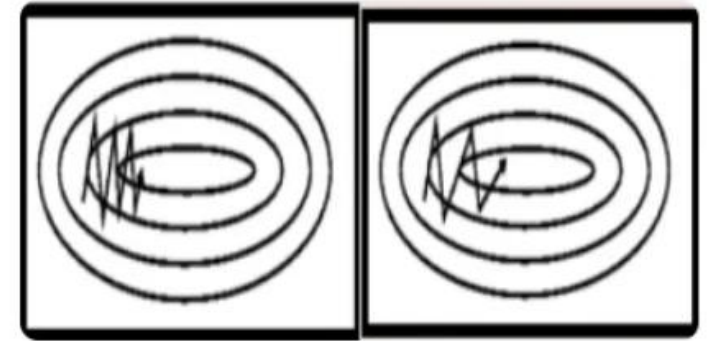
$$\theta_{t+1} = \theta_t - \eta g_t$$

La actualización actual ya
no depende solamente del
gradiente actual.

Velocidad

$$v_t = \beta v_{t-1} + \eta g_t$$

$$\theta_{t+1} = \theta_t - v_t$$



Not only move in the (opposite)
direction of the gradient, but also
move in the "averaged" direction of
the last few updates

Actualización del
pasado

Contribución
gradiente actual

$$v_t = \boxed{\beta v_{t-1}} + \boxed{\eta g_t}$$

momentum $0 \leq \beta < 1$

Momentum

Velocidad

$$v_t = \beta v_{t-1} + \eta g_t$$

$$\theta_{t+1} = \theta_t - v_t$$

Suponiendo que $\rightarrow \beta = 0.9$.

$$v_t = 0.9v_{t-1} + \eta g_t$$

$$v_{t-1} = 0.9v_{t-2} + \eta g_{t-1}$$

Luego

$$v_t = 0.9(0.9v_{t-2} + \eta g_{t-1}) + \eta g_t$$

$$v_t = 0.9\eta g_{t-1} + 0.9^2 v_{t-2} + \eta g_t$$

Finalmente

$$v_t = \eta g_t + \beta \eta g_{t-1} + \beta^2 \eta g_{t-2} + \beta^3 \eta g_{t-3} + \dots$$

Often referred to as "velocity" v

"velocity" from the previous iteration

$$\Delta w_{i,j}(t) := \alpha \cdot \Delta w_{i,j}(t-1) + \eta \cdot \frac{\partial \mathcal{L}}{\partial w_{i,j}}(t)$$

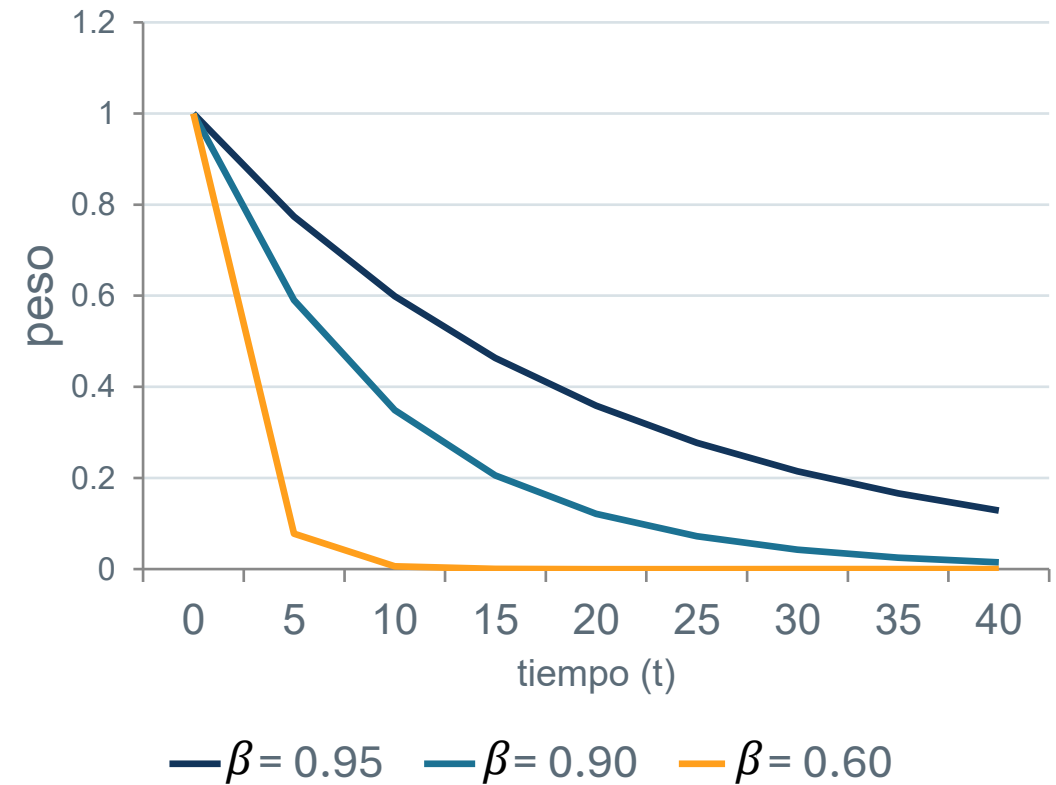
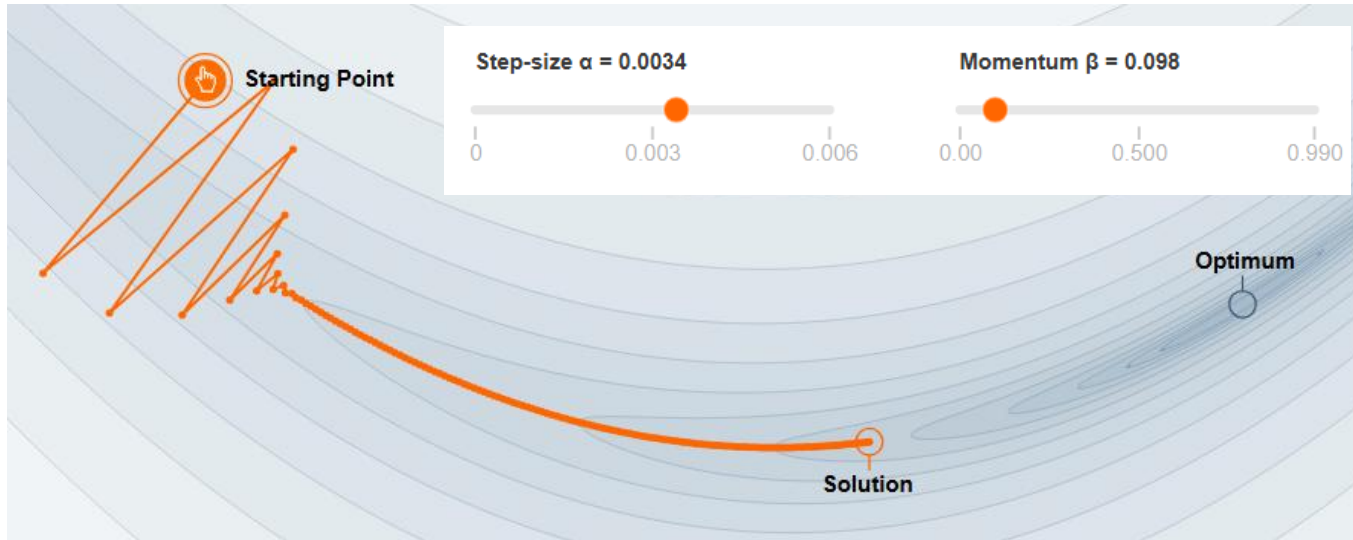
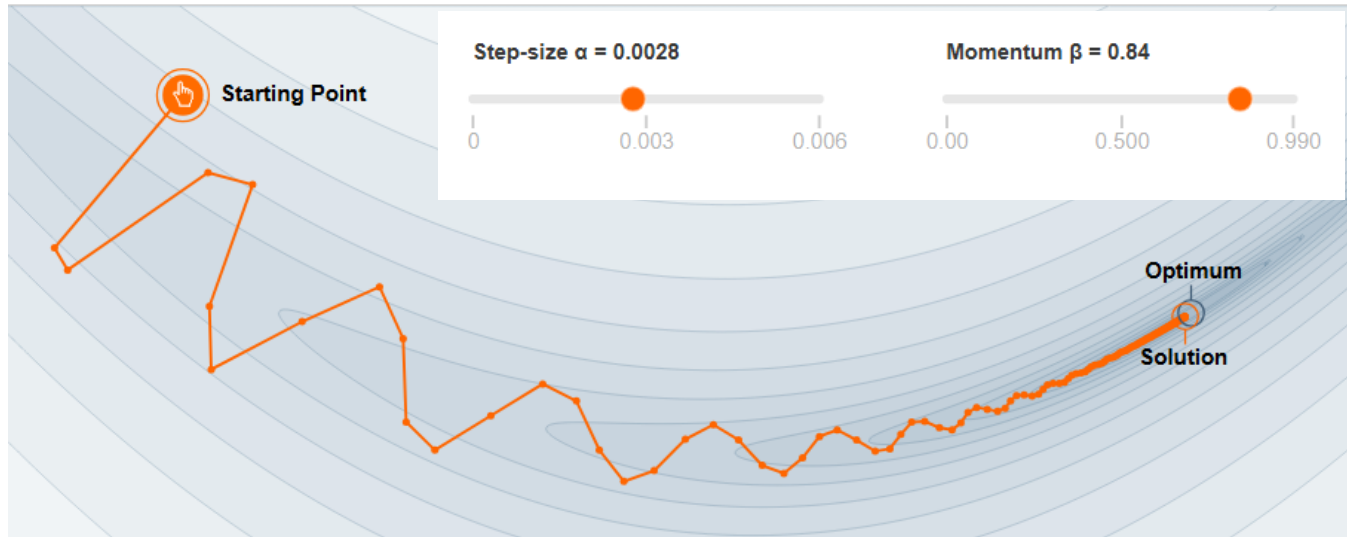
Usually, we choose a momentum rate between 0.9 and 0.999; you can think of it as a "friction" or "dampening" parameter

Regular partial derivative/gradient multiplied by learning rate at current time step t

Weight update using the velocity vector:

$$w_{i,j}(t+1) := w_{i,j}(t) - \Delta w_{i,j}(t)$$

Momentum



[Link](#)

AdaGrad

¿Todos los parámetros deberían ser entrenados con el mismo learning rate?

Acumulando el cuadrado de los gradientes es posible:

$$\boxed{s_t = s_{t-1} + g_t \odot g_t} \Rightarrow g_t \odot g_t = g_t^2 \Rightarrow g_t = \begin{bmatrix} g_{1,t} \\ g_{2,t} \\ \vdots \\ g_{n,t} \end{bmatrix} \quad g_t^2 = \begin{bmatrix} g_{1,t}^2 \\ g_{2,t}^2 \\ \vdots \\ g_{n,t}^2 \end{bmatrix}$$

$$g_t = \nabla_{\theta} L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \boxed{\frac{\eta}{\sqrt{s_t} + \epsilon} g_t}$$

Learning rate
efectivo

¡CUIDADO! AdaGrad acumula los gradientes cuadrados y, con el tiempo, el denominador puede crecer demasiado, reduciendo excesivamente el tamaño de los pasos.

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$$

$$|g_{1,t}| \ll \Rightarrow \text{luego } g_{1,t}^2 \ll$$

$$|g_{2,t}| \gg \Rightarrow \text{luego } g_{2,t}^2 \gg$$

$$s_{1,t} \downarrow \Rightarrow \frac{\eta}{\sqrt{s_{1,t}} + \epsilon} \uparrow$$

$$s_{2,t} \uparrow \Rightarrow \frac{\eta}{\sqrt{s_{2,t}} + \epsilon} \downarrow$$

RMSprop

¿Es necesario recordar todos los gradientes desde el inicio del entrenamiento?

Reemplazar la suma por un promedio móvil exponencial del gradiente al cuadrado.

$$s_t = s_{t-1} + g_t \odot g_t$$

Primera iteración

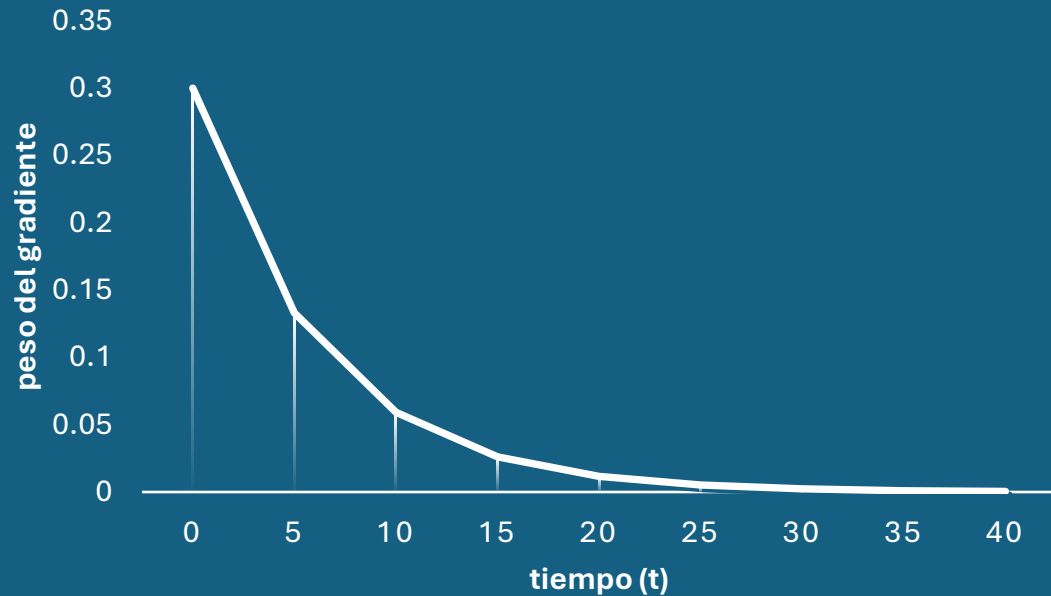
$$s_1 = s_0 + g_1 \odot g_1 \rightarrow s_1 = g_1^2$$

Segunda iteración

$$s_2 = s_1 + g_2 \odot g_2 \rightarrow s_2 = g_1^2 + g_2^2$$

Tercera iteración

$$s_3 = s_2 + g_3 \odot g_3 \rightarrow s_3 = g_1^2 + g_2^2 + g_3^2$$



Ventana de memoria efectiva de RMSprop

¿Es necesario recordar todos los gradientes desde el inicio del entrenamiento?

Reemplazar la suma por un promedio móvil exponencial del gradiente al cuadrado.

$$s_t = s_{t-1} + g_t \odot g_t \quad \Rightarrow \quad s_t = \beta s_{t-1} + (1 - \beta) g_t \odot g_t$$

$$s_t = \beta s_{t-1} + (1 - \beta) g_t^2$$

$$s_t = \beta s_{t-1} + (1 - \beta) g_t^2$$

$$s_{t-1} = \beta s_{t-2} + (1 - \beta) g_{t-1}^2 \quad \Rightarrow \quad s_t = \beta(\beta s_{t-2} + (1 - \beta) g_{t-1}^2) + (1 - \beta) g_t^2$$

$$s_t = \beta^2 s_{t-2} + \beta(1 - \beta) g_{t-1}^2 + (1 - \beta) g_t^2$$

$$s_t = (1 - \beta) g_t^2 + \beta(1 - \beta) g_{t-1}^2 + \beta^2 s_{t-2}$$

$$s_t = (1 - \beta)[g_t^2 + \beta g_{t-1}^2 + \beta^2 g_{t-2}^2 + \dots]$$

Gradientes antiguos tienen menos peso

Adam = Momentum-like+ RMSprop-like



Momentum recuerda hacia dónde hemos estado avanzando; RMSprop recuerda qué tan grandes han sido recientemente los gradientes.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Mantiene información sobre la magnitud reciente de los gradientes

Historia Gradiente
 actual

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Captura la dirección y suaviza gradientes

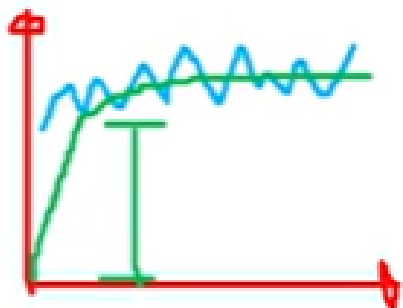
Momentum + adaptación: el optimizador por defecto en Deep Learning

ADAM → Adaptive Moment Estimation

ADAM → Adaptive Moment Estimation

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$



En las primeas épocas no existe el **historial de gradientes anteriores** para construir los promedios móviles de Adam

$$m_0 = 0 \quad v_0 = 0$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^2}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^2}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Adaptive Learning Rate via ADAM

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

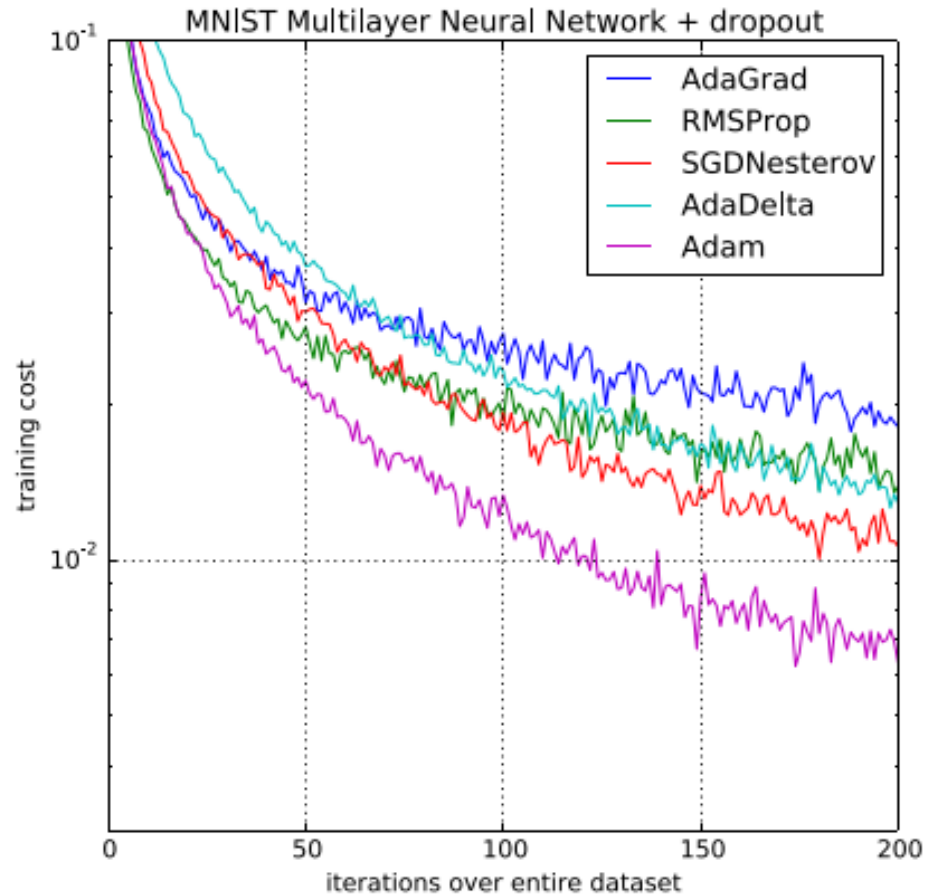
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

Also add a bias correction term
for better conditioning in earlier iterations

Optimizadores



Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. <https://arxiv.org/abs/1412.6980>

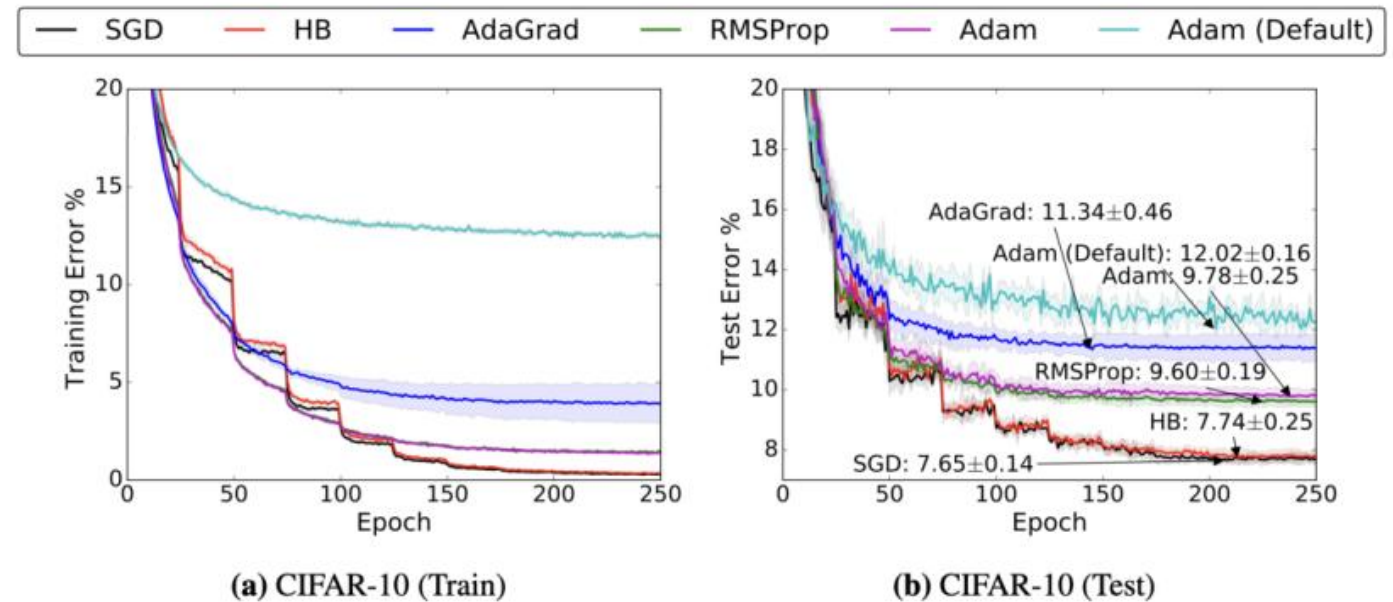


Figure 1: Training (left) and top-1 test error (right) on CIFAR-10. The annotations indicate where the best performance is attained for each method. The shading represents $\pm$ one standard deviation computed across five runs from random initial starting points. In all cases, adaptive methods are performing worse on both train and test than non-adaptive methods.

Wilson AC, Roelofs R, Stern M, Srebro N, Recht B. The marginal value of adaptive gradient methods in machine learning, <https://arxiv.org/abs/1705.08292>

Redes Convolucionales

Valores por defecto que casi nunca hay que tocar:

 β_1

0.9

 β_2

0.999

 ε

1e-8

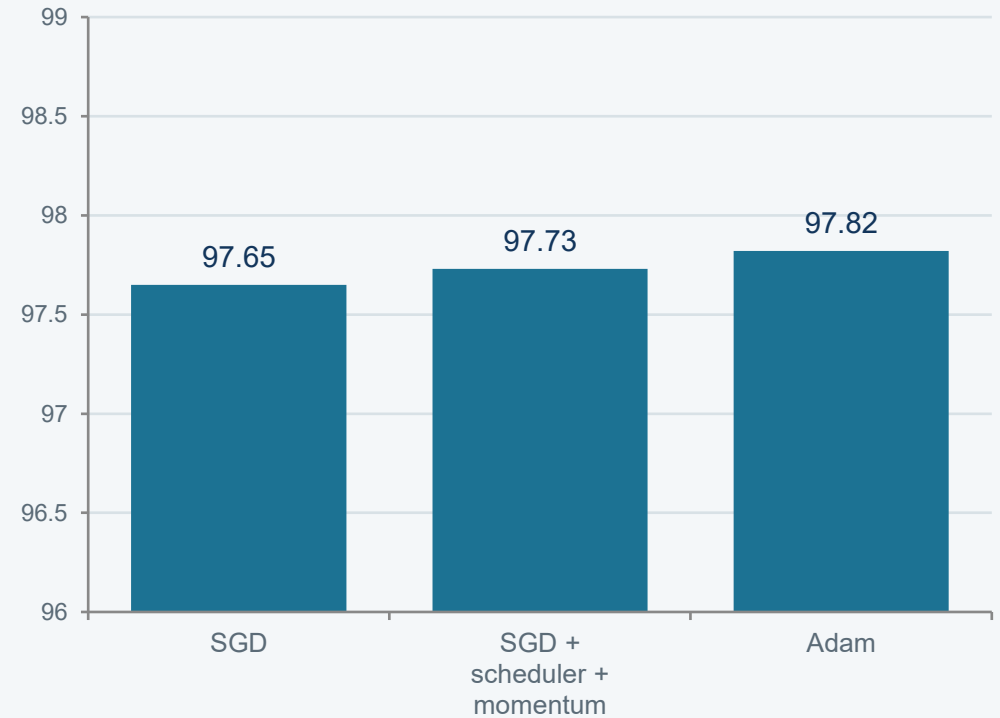
 η

1e-3

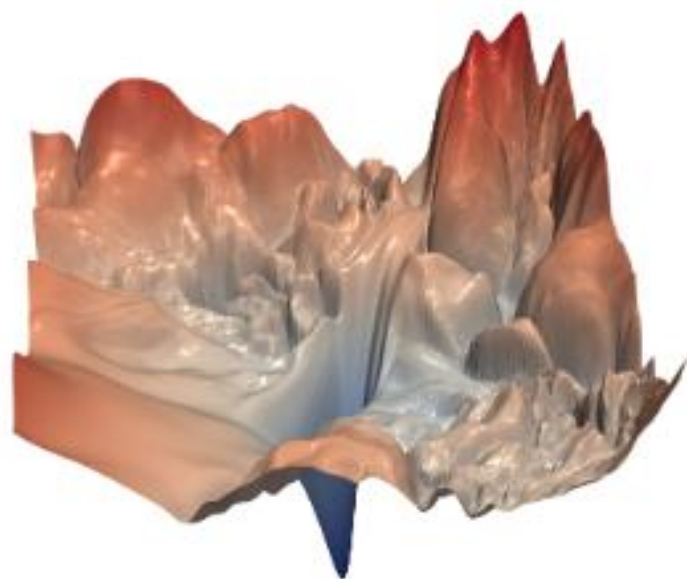
Uso en PyTorch:

```
opt = torch.optim.SGD(  
    model.parameters(), lr=0.1, momentum=0.9)  
  
opt = torch.optim.Adam(  
    model.parameters(), lr=0.001)
```

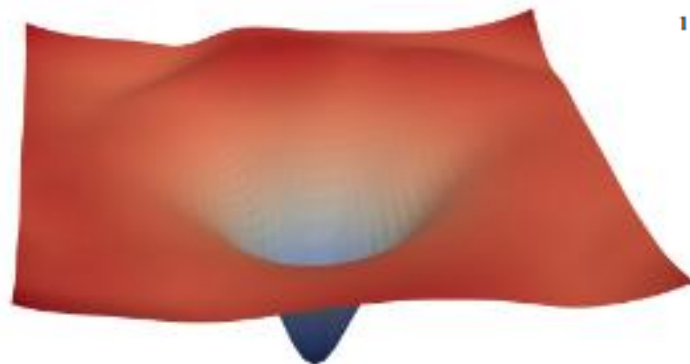
MNIST · MLP · 100 épocas



¿Por qué es útil ?



(a) without skip connections



(b) with skip connections

Visualizing the Loss Landscape of Neural Nets

Hao Li¹, Zheng Xu¹, Gavin Taylor², Christoph Studer³, Tom Goldstein¹

¹University of Maryland, College Park ²United States Naval Academy ³Cornell University
{haoli,xuzh,tong}@cs.umd.edu, taylor@usna.edu, studer@cornell.edu

Sin skip connections: la superficie de pérdida es mucho más irregular, con regiones abruptas, picos, valles estrechos y cambios fuertes de curvatura. Eso hace que la optimización por gradiente sea más difícil.

Con skip connections: la superficie se vuelve mucho más suave y aparece una cuenca amplia alrededor del mínimo. Por tanto, el gradiente cambia de manera más estable y el optimizador puede avanzar con mayor facilidad.